

DESIGN_KIT.md — E.Y.E.S. Frontend Design System

Reusable visual language for this dashboard: color tokens, component recipes, and copy-paste-ready CSS/JSX. Read this **before** styling a new panel or adding a new UI pattern — almost everything you need already exists somewhere in `App.css` or `src/components/`. The goal is that a new feature looks like it was built by the same hand as everything else, without re-deriving the same button/table/chip styles from scratch each time.

This file documents *what exists and how to reuse it*. It does not duplicate business-logic gotchas — those live in `SKILLS.md`.

The PDF is generated from this file. After editing, run `node scripts/build-design-kit-pdf.mjs` to refresh `DESIGN_KIT.pdf` so the two never drift apart.

0. How to use this file

1. **Building a panel?** → jump to §7 Recipe: building a new panel.
 2. **Need a color?** → §1 Design tokens. Never hardcode a hex value in a component — always reference a `var(--token)`, so it follows the active theme automatically.
 1. **Need a button/table/chip/badge?** → §3 through §6 are copy-paste recipes with the exact class names already in `App.css`.
 2. **Need pagination?** → §8 Pagination — a `Pager` component and `usePagination` hook already exist; never hand-roll page state again.
 1. **Adding a genuinely new pattern?** Add it to `App.css` following the `.block__element--modifier` naming already in use (see §9), then add a short entry here so the next feature reuses it instead of reinventing it.
-

1. Design tokens

Every color is a CSS custom property scoped to `.app-shell`, redefined per theme. **Never hardcode a color** — always `var(--token-name)`. There are three themes: the OS-preference default (light, or dark via `prefers-color-scheme`), an explicit `data-theme="dark"`, and the brand theme `data-theme="eko"` (warm/amber). All three define the *same* token names — that's what makes a component theme-agnostic as long as it only uses tokens.

TOKEN	LIGHT (DEFAULT)	DARK	EKO (BRAND)	USE FOR
<code>--page</code>	<code>#f9f9f7</code>	<code>#0d0d0d</code>	<code>#fffd8</code>	Page background
<code>--surface-1</code>	<code>rgba(252, 252, 251, .75)</code>	<code>rgba(26, 26, 25, .65)</code>	<code>rgba(255, 255, 255, .85)</code>	Card/panel background (always paired with <code>backdrop-filter: blur(.</code>
<code>--text-primary</code>	<code>#0b0b0b</code>	<code>#ffffff</code>	<code>#4a4b4d</code>	Headings, primary text, values
<code>--text-secondary</code>	<code>#52514e</code>	<code>#c3c2b7</code>	<code>#6f7174</code>	Labels, descriptions
<code>--muted</code>	<code>#898781</code>	<code>#898781</code>	<code>#93959a</code>	Timestamps, hints, disabled-ish
<code>--border</code>	<code>rgba(11, 11, 11, .08)</code>	<code>rgba(255, 255, 255, .08)</code>	<code>rgba(74, 75, 77, .12)</code>	Card/input borders
<code>--gridline</code>	<code>#e1e0d9</code>	<code>#2c2c2a</code>	<code>#f0e6d4</code>	Table row dividers, chart grid
<code>--baseline</code>	<code>#c3c2b7</code>	<code>#383835</code>	<code>#e2d3b8</code>	Neutral chip border-left, chart baseline
<code>--row-stripe</code>	<code>rgba(11, 11, 11, .02)</code>	<code>rgba(255, 255, 255, .03)</code>	<code>rgba(245, 166, 35, .05)</code>	Zebra-striped table rows, subtle
<code>--series-1</code>	<code>#2a78d6</code>	<code>#3987e5</code>	<code>#cf7d00</code>	Primary accent — active states, primary chart series
<code>--hover-wash</code>	<code>rgba(42, 120, 214, .08)</code>	<code>rgba(57, 135, 229, .12)</code>	<code>rgba(245, 166, 35, .12)</code>	Hover backgrounds
<code>--good</code>	<code>#006300</code>	<code>#0ca30c</code>	<code>#1f7a3d</code>	Positive values, success pills
<code>--good-wash</code>	<code>rgba(12, 163, 12, .12)</code>	<code>rgba(12, 163, 12, .18)</code>	<code>rgba(31, 122, 61, .12)</code>	Success pill/badge background
<code>--warning</code>	<code>#8a5a00</code>	<code>#fab219</code>	<code>#8a5a00</code>	At-risk / partial states
<code>--warning-wash</code>	<code>rgba(250, 178, 25, .16)</code>	<code>rgba(250, 178, 25, .18)</code>	<code>rgba(245, 166, 35, .2)</code>	Warning banner/pill background
<code>--critical</code>	<code>#d03b3b</code>	<code>#e66767</code>	<code>#c0392b</code>	Failures, critical tier, destructive actions
<code>--critical-wash</code>	<code>rgba(208, 59, 59, .12)</code>	<code>rgba(230, 103, 103, .18)</code>	<code>rgba(192, 57, 43, .1)</code>	Failure pill/banner background

Eko theme also defines `--brand-amber: #f5a623` and `--accent-ink: #a35f00` (the raw, uncorrected brand amber — only safe for chip *selected* states where dark text sits on top; it fails contrast as a chart mark, which is why `--series-1` uses a darker step instead).

Rule of thumb: value/foreground color = a plain token (`--good`, `--critical`, `--series-1`); background = the matching `--wash` token. Never pair `--good` foreground with `--critical-wash` background, etc.

Semantic direction, not literal color names: `--good` / `--critical` mean "the direction that's actually good/bad for THIS metric," not universally "green/red." E.g. on the Inactivity Rate KPI card, a rate going *up* is bad (critical) and going *down* is good — the CSS classes are literally named `.kpi-card-mock__trend-val--up { color: var(--critical); }` (see the comment in `App.css`: "*in inactivity, up is bad*"). Pick the class by what the number means, not by whether it increased.

2. Layout primitives

Panel — the universal card wrapper

Every section on the dashboard is a `<Panel>` (`src/components/Panel.tsx`). It gives you: title/subtitle header, an optional collapse toggle (on by default), and an optional full-screen "focus mode" toggle for review meetings. Never build a bare `<div className="panel">` by hand — always go through the component so collapse/focus behavior stays consistent.

```
import Panel from "./Panel";

<Panel
  title="Section title"
  subtitle="One line of context - what this shows and why"
  focusable           // opt in only for chart/table-shaped panels
  headerExtra={<SomeToggleButtons />} // e.g. chart-controls, see §3
>
  {/* content */}
</Panel>
```

CSS: `.panel` (the card — surface, border, blur, shadow, hover lift), `.panel__header`, `.panel__header-titles`, `.panel__subtitle`, `.panel__header-actions`, `.panel__icon-btn` (the collapse/focus icon buttons), `.panel--focused` + `.panel-focus-backdrop` (full-screen mode).

Grids

- `.dashboard-grid-2col` — two panels side by side, wraps on narrow screens (used to pair related panels, e.g. Daily Changes + Adaptive Tuning).
- `.kpi-grid-4col` — the 4 top-level KPI cards row.
- `.stat-row` + `.stat-tile` — a simpler stat grid (`repeat(auto-fit, minmax(180px,1fr))`) for panels that need stat boxes but not the full KPI-card treatment. Add `.stat-tile--clickable` for a stat that opens a drill-down.

Tables

Every data table in the app uses the same two classes together:

```
<div className="table-scroll">
  <table className="data-table">
    <thead><tr><th>Col</th></tr></thead>
    <tbody>
      <tr><td className="num">123</td></tr>
    </tbody>
  </table>
</div>
```

- `.table-scroll` — `overflow-x: auto`, so a wide table scrolls inside its own box instead of blowing out the page.
- `.data-table` — zebra-striped (`tbody tr:nth-child(even)`), uppercase muted headers, `border-bottom: 1px solid var(--gridline)` rows.
- `td.num` — `font-variant-numeric: tabular-nums`, use on every numeric column so digits align.

3. Buttons

CLASS	WHEN TO USE
<code>.trigger-button</code>	Default action button — neutral surface, border, hover lift. This is the base class; combine with a modifier below or leave bare for a neutral action (Save, Discard, etc.).
<code>.trigger-button--confirm</code>	Destructive / sends-something-real actions only — solid <code>--critical</code> background. Reserved for things like "Run alerts now" that message real people. Do NOT use this for an ordinary Save button — use the shared <code>.form-save</code> (§6) instead.
<code>.link-button</code>	Inline text-styled action inside a table row (e.g. "Recipients" / "Hide" expand toggle, "Discard edit", "Reset to default").
<code>.panel__icon-btn</code>	Small icon-only button inside a panel header (collapse, focus).

```
<button type="button" className="trigger-button" onClick={save}>Save</button>
<button type="button" className="trigger-button trigger-button--confirm" onClick={runNow}>Run alerts now</button>
<button type="button" className="link-button" onClick={toggle}>Details</button>
```

Disabled state is automatic: `.trigger-button:disabled { opacity: .5; cursor: not-allowed; }`.

4. Chips & pills

Filter chip — a toggleable pill filter

```
<button className={`filter-chip${active ? " filter-chip--selected" : ""}`}>
  Label <span className="roster-tab__count">42</span>
</button>
```

Selected state fills solid `--series-1` with white text. Use `.filter-chip--all` for an "All" chip that should look distinct (dashed border, bold).

Tier chip — the escalation-tier stat chip

```
<button className={`tier-chip tier-chip--${tier} tier-chip--clickable`}>
  <span className="tier-chip__count">38</span>
  <span className="tier-chip__range">3-7d</span>
</button>
```

`tier-chip--self|breach|escalated|critical` each set a different `border-left` color (warning / series-1 / orange / critical). **Do not** reintroduce a tier *name* label here (Self-nudge / RM follow-up / etc.) — a past redesign deliberately replaced tier names with day-ranges only across the whole frontend; keep new tier UI consistent with that (see `KpiPanel.tsx`, `DailyChangesPanel.tsx`).

Range chip

Same idea as tier chip but for inactivity-day buckets (`3-7`, `7-15`, ...); see `.range-chip`, `.range-chip--active`, `.range-chip--ignored` (dims a bucket that's excluded from the headline rate, e.g. 90+/unmeasurable).

Status pill — delivered/partial/failed/unknown

```
<span className={`status-pill ${meta.className}`}>
  <span aria-hidden="true">{meta.icon}</span> {meta.label}
</span>
```

Classes: `status--delivered` (good), `status--partial` (warning), `status--failed` (critical), `status--unknown` (muted/neutral). All four are defined alongside `STATUS_META` in `statusOptions.ts` — look that map up and add a case to it rather than inventing a new ad-hoc pill.

5. Forms

Select / input (the "glass" filter bar look)

```
<select className="glass-select"> ... </select>
<input className="glass-input" />
```

Always pair with the `option` fix — native `<option>` popups are NOT styled by the app's CSS and default to a light background regardless of theme. Every themed `<select>` needs:

```
.your-select option { background: var(--page); color: var(--text-primary); }
```

(`App.css` already does this for `.glass-select`, `.roster-range__select`, `.entity-filter select` — copy that pattern for any NEW select, or dark-theme option text will be invisible. This was a real bug once — don't reintroduce it.)

Toggle switch (on/off, e.g. Settings kill switches)

```
<label className="setting-toggle">
  <input type="checkbox" className="setting-toggle__input" checked={v} onChange={ ... } />
  <span className="setting-toggle__track" aria-hidden="true">
    <span className="setting-toggle__thumb" />
  </span>
  <span className={`setting-toggle__state${v ? " setting-toggle__state--on" : ""}`}>
    {v ? "On" : "Off"}
  </span>
</label>
```

The real `<input type="checkbox">` stays visually hidden but functional (keyboard/focus-accessible) — never swap it for a plain `<div onClick>`.

Card-with-input (settings / template editors)

`.setting-card` — a bordered, blurred card holding one labeled control (number input, toggle, or textarea). `.setting-card--off` reddens the card when a toggle inside it is in a "this is disabled and that matters" state. `.template-card` is the same idea specialized for a textarea + live preview (see `TemplatesPanel.tsx` for the fullest example: label, description, textarea, required-placeholder chips, preview box, discard/reset actions).

6. Feedback & banners

CLASS	USE
<code>.empty-state</code>	Centered, muted "nothing here" message inside a panel body.
<code>.empty-state--muted</code>	A smaller, quieter empty-state line (e.g. "None today.")
<code>.error-banner</code>	A hard failure (API unreachable). Full-width, at the top of <code>app-main</code> .
<code>.channels-off-banner</code>	Shell-level warning banner shown on every tab when a kill switch is off — pattern to reuse for any "this thing is globally disabled and it matters everywhere" state.
<code>.settings-warning</code>	Inline warning INSIDE a settings/config panel (e.g. "both channels are off").
<code>.form-feedback--ok</code> / <code>--error</code>	Save-result text next to a Save button (green/red).
<code>.template-card__field-error</code>	Inline validation error under one specific field.

The shared save row — `.form-*`

Any panel that edits config uses the SAME save row. Used today by `SettingsPanel.tsx` and `TemplatesPanel.tsx`:

```

<div className="form-actions">
  <button
    type="button"
    className={`trigger-button form-save${saved && !dirty ? " form-save--ok" : ""}`}
    disabled={saving || !dirty}
  >
    {saving && <span className="form-spinner" aria-hidden="true" />}
    {saved && !dirty && <span aria-hidden="true">OK </span>}
    {saving ? "Saving..." : saved && !dirty ? "Saved" : "Save changes"}
  </button>
  {message && (
    <span className={`form-feedback form-feedback--${isError ? "error" : "ok"}} role="status">
      {message}
    </span>
  )}
</div>

```

State machine: idle → saving (spinner) → saved (green) / error (red message). Any edit clears a previous saved/error verdict back to idle, so a success tick never lingers next to a value the server doesn't have yet.

These are named `form-*`, **not** `settings-*`, on purpose. They were originally `.settings-save` / `.settings-actions` / `.settings-feedback` / `.settings-spinner`, which implied they belonged to the Settings panel; when `TemplatesPanel` reused them there were two `.settings-save` buttons in the DOM and `document.querySelector(".settings-save")` silently returned the wrong one. If you build a third config panel, reuse `.form-*` — don't invent `.mypanel-save`, and don't re-prefix a shared pattern with one panel's name (§9). The still- `settings-` -prefixed classes (`.settings-grid`, `.settings-warning`, `.settings-reset`) are genuinely Settings-only.

7. Recipe: building a new panel

1. Fetch data in a `useEffect`, store in `useState<T | null>` + an `error` boolean flag. On error, `return null` (fail silent, not a broken card — this dashboard's convention: a panel that can't load simply doesn't render, it doesn't show a scary red box unless it's a top-level page error).
2. Wrap everything in `<Panel title="..." subtitle="...">`.
3. If the panel shows a list that could ever exceed ~15-25 rows, **use** `usePagination` + `Pager` **from day one** (§8) — don't ship a silent `.slice(0, N)` truncation. That was a real bug fixed across 8 panels once; don't reintroduce the pattern.
4. Use `.table-scroll > table.data-table` for tabular data, or the chip/list

patterns above for compact summaries.

1. Any color choice → a token from §1, chosen by *meaning* (good/warning/critical), never a raw hex.
2. Any select/input → `.glass-input` / `.glass-select` + the `option` fix if it's a select.
3. Editing config? Reuse the `.form-*` save row (§6) rather than styling a new one.
4. Mount it in `App.tsx` under the right tab (`overview` / `insights` / `system` / `analytics`) alongside thematically similar panels.
5. Run `npm tsc --noEmit` and `npm run build` before calling it done.

8. Pagination

Never hand-roll page state. Two reusable pieces already exist:

`src/usePagination.ts` — takes an array + page size, returns the current page's slice and safe page-clamping (so a filter change that shrinks the array can never leave you stuck on an empty out-of-range page):

```

import { usePagination } from "../usePagination";

const { page, pageCount, visible, setPage, resetPage } = usePagination(items, 25);

```

`src/components/Pager.tsx` — the prev/next control, renders nothing when there's only one page:

```
import Pager from "../Pager";

<Pager
  page={page}
  pageCount={pageCount}
  visibleCount={visible.length}
  totalCount={items.length}
  onPrev={() => setPage(page - 1)}
  onNext={() => setPage(page + 1)}
/>
```

Call `resetPage()` whenever the underlying array is swapped for a **DIFFERENT query**, not just filtered shorter — e.g. switching an RM/DC toggle, or a "card click" panel that receives a wholly new `records` prop when a different filter is picked. Do this in a `useEffect` keyed on whatever identifies the new query (a title string, a role toggle, etc.):

```
useEffect(() => { resetPage(); }, [roleOrTitleOrWhateverIdentifiesTheQuery]);
```

Reference implementations: `AllCspsTable.tsx` (search + status + range filters, resets on any filter change), `FilteredResultsPanel.tsx` (resets on `title` change), `GeoHeatMap.tsx` / `RmDcPerformancePanel.tsx` (resets on a dimension/role toggle).

9. Naming convention

BEM-ish, already consistent across the whole file — follow it for anything new:

<code>.block</code>	→ the component root (<code>.template-card</code>)
<code>.block__element</code>	→ a part of it (<code>.template-card__label</code>)
<code>.block--modifier</code>	→ a state/variant of the block (<code>.template-card--error</code>)
<code>.block__element--modifier</code>	→ a state of a specific part (<code>.kpi-card-mock__trend-val--up</code>)

Prefer a NEW block name over overloading an existing one for an unrelated purpose (e.g. `reach-stat-card` got its own block rather than reusing `kpi-card-mock`, because the latter is wired to App.tsx-specific onClick handlers and hardcoded card identities).

And when a pattern is genuinely shared by two or more panels, give it a **neutral** block name rather than one panel's prefix — see the `.form-*` note in §6 for what goes wrong otherwise.

10. Icons

Emoji, not an icon font/library — kept intentionally simple and consistent with the existing header icons. Current vocabulary (reuse before picking a new one): chart-down for rate/trend, people for RM groups, warning triangle for alerts, siren for escalation/critical, speech balloon for WhatsApp/CSP messaging, bust for RM, office-worker for DC, memo for templates/editing, gear for settings, bell-with-slash for a muted/disabled channel, tick/cross for success/failure, test-tube for diagnostics.

11. File map

WHAT	WHERE
Design tokens + every component's CSS	<code>frontend/src/App.css</code>
Panel wrapper	<code>frontend/src/components/Panel.tsx</code>
Pagination hook	<code>frontend/src/usePagination.ts</code>
Pagination control	<code>frontend/src/components/Pager.tsx</code>
Status pill metadata	<code>frontend/src/statusOptions.ts</code>
Range bucket labels/logic	<code>frontend/src/rangeOptions.ts</code>
Tier logic (frontend mirror)	<code>frontend/src/tierOptions.ts</code>
Theme switcher	<code>frontend/src/components/ThemeSwitcher.tsx</code> , <code>frontend/src/useTheme.ts</code>
This doc's PDF builder	<code>scripts/build-design-kit-pdf.mjs</code>

Business-logic/architecture conventions (not visual) live in `SKILLS.md` — read both when building something that's both a new data flow AND a new UI.